```
import java.util.*;

public class SortingAlgorithms {

    // Merge Sort
    static void mergeSort(int[] arr, int l, int r) {
        if (l < r) {
            int m = (l + r) / 2;
            mergeSort(arr, l, m);
            mergeSort(arr, m + 1, r);
            merge(arr, l, m, r);
        }
    }

    static void merge(int[] arr, int l, int m, int r) {
        int n1 = m - l + 1;
        int n2 = r - m;

        int[] L = new int[n1];
        int[] R = new int[n2];

        for (int i = 0; i < n1; i++)
            L[i] = arr[l + i];

        for (int j = 0; j < n2; j++)
            R[j] = arr[m + 1 + j];

        int i = 0, j = 0, k = l;

        while (i < n1 && j < n2) {
            if (L[i] <= R[j])
                arr[k++] = L[i++];
            else
                arr[k++] = R[j++];
        }

        while (i < n1)
            arr[k++] = L[i++];

        while (j < n2)
            arr[k++] = R[j++];
    }

    // Quick Sort
```

```
static void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

static int partition(int[] arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

// Heap Sort
static void heapSort(int[] arr) {
    int n = arr.length;

    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}
```

```
static void heapify(int[] arr, int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        int swap = arr[i];
        arr[i] = arr[largest];
        arr[largest] = swap;

        heapify(arr, n, largest);
    }
}

// Counting Sort
static void countingSort(int[] arr) {
    int max = Arrays.stream(arr).max().getAsInt();

    int[] count = new int[max + 1];

    for (int num : arr)
        count[num]++;

    int index = 0;

    for (int i = 0; i <= max; i++) {
        while (count[i]-- > 0) {
            arr[index++] = i;
        }
    }
}

public static void main(String[] args) {

    int[] data = {38200, 12500, 7800, 54000, 23100,
            9400, 61000, 4500, 31700, 18900};
```

```
    System.out.println("=== FinPulse - Advanced Sorting & Data Ranking ===");
    System.out.println("Dataset: 10 customer transaction records");
    System.out.println("Input: " + Arrays.toString(data));

    // Merge Sort
    int[] mergeArr = data.clone();
    mergeSort(mergeArr, 0, mergeArr.length - 1);

    System.out.println("\n1. MERGE SORT");
    System.out.println("Sorted: " + Arrays.toString(mergeArr));
    System.out.println("Time: O(n log n) | Space: O(n)");

    // Quick Sort
    int[] quickArr = data.clone();
    quickSort(quickArr, 0, quickArr.length - 1);

    System.out.println("\n2. QUICK SORT");
    System.out.println("Sorted: " + Arrays.toString(quickArr));
    System.out.println("Time: O(n log n) avg | Space: O(log n)");

    // Heap Sort
    int[] heapArr = data.clone();
    heapSort(heapArr);

    System.out.println("\n3. HEAP SORT");
    System.out.println("Sorted: " + Arrays.toString(heapArr));

    System.out.println("Top-3 High Value Alerts:");
    for (int i = heapArr.length - 1; i >= heapArr.length - 3; i--) {
        System.out.print("Rs." + heapArr[i] + " ");
    }
    System.out.println("\nTime: O(n log n) | Space: O(1)");

    // Counting Sort
    int[] txnIds = {5, 3, 8, 1, 9, 2, 7, 4, 6, 0};

    countingSort(txnIds);

    System.out.println("\n4. COUNTING SORT");
    System.out.println("Sorted TXN IDs: " + Arrays.toString(txnIds));
    System.out.println("Time: O(n+k) | Space: O(k)");

    System.out.println("\nBUILD SUCCESSFUL");
}
```

```
}
```